RT KNITS · INNOVATION CHALLENGE

Design an Agentic AI-Powered CMMS

for Smart Manufacturing

Reimagine how a 24/7 factory keeps its machines running — with AI agents that talk, plan, and coordinate.

The Host	The Problem	The Mission
RT Knits · Mauritius · ~1,700 people	A critical CMMS that's stuck at a desktop PC	Replace the coordinator bottleneck with an AI agent

The scene

It's 2 a.m. on the knitting floor in Mauritius. A circular knitter — one of hundreds of machines running 24/7 — has just seized a needle. **The operator knows how to spot the problem. She just doesn't know how to get the right technician, with the right parts, to the right machine, fast enough to save the shift.**

Today, she walks to a desktop PC, logs in, fills out a request, and waits for a coordinator to route it. Tomorrow — if you win this challenge — she sends a WhatsApp voice note. An AI agent understands it, asks for a photo, figures out it's a mechanical fault, checks who's free, and dispatches the right technician within minutes.

That's the prize: **more machines running, fewer humans stuck coordinating, a factory that learns from every fix.**

About RT Knits

RT Knits is a vertically integrated garment manufacturer based in Mauritius, employing around 1,700 people. The factory runs 24/7 with highly automated machinery, and is recognized as a leader in innovation and sustainability in textile manufacturing.

Why uptime is sacred:

- On-time delivery to global customers depends on it.
- Production efficiency — and the unit economics of every garment — depends on it.
- Every minute of unplanned downtime is cost that can't be recovered.

Where we are today

RT Knits runs a homegrown CMMS (Computerized Maintenance Management System) that has served us well — but is now the bottleneck.

- Requests must be submitted from a desktop PC.
- Work tracking relies on RFID-based timestamps.
- Planning and prioritization live in the head of one central coordinator.
- There's little mobility, limited real-time interaction, and a lot of manual decision-making.

The maintenance team

- 25 technicians across multiple trades — mechanics, electricians, welders, and more.
- 1 planner / coordinator.
- 1 head of maintenance.

Your challenge

Design a next-generation CMMS powered by agentic AI — a system that acts as a digital maintenance coordinator, so humans can focus on execution instead of coordination.

Concretely, we want a system that:

- Automates maintenance request handling end-to-end.
- Responds faster and prioritizes smarter than a human-only workflow can.
- Reduces the load on the central coordinator.
- Captures richer data for continuous improvement.
- Operates seamlessly through mobile-first interfaces — starting with WhatsApp.

Why this matters

Maintenance inefficiency is not an abstract KPI. It directly affects:

- Production downtime — lost hours, late shipments, unhappy customers.
- Employee safety — a delayed fix on a hazard is a real risk.
- Operating cost — every minute of delay compounds.

A smarter CMMS can: lift machine uptime, shrink response time, improve the quality of repairs, and make data-driven decisions the default rather than the exception.

Why now: the agentic AI opportunity

Modern agentic LLMs can do things that weren't possible even two years ago, and that's what makes this project interesting today:

- **Talk naturally** — in text and voice, in multiple languages.
- **See** — interpret photos of defects, machine issues, parts.
- **Decide** — triage, prioritize, assign, and reschedule based on context.
- **Coordinate** — orchestrate multi-step workflows with humans in the loop at the right moments.

Your challenge is to take these capabilities and make them useful on an actual factory floor, for actual workers, with actual machines. Not a demo. A system.

Who this challenge is for

This is a systems problem at the intersection of hardware, software, and human behavior. No single discipline can solve it alone — which is why the best teams will be cross-functional.

Engineering & Mechatronics	Computer Science & AI	Business & Design
You get the factory Model real constraints: shift patterns, machine taxonomies, trades, RFID, geolocation, sensor integration, preventive maintenance logic.	**You build the agent** Design the multimodal LLM stack, tool use, memory, prioritization logic, planning, human-in-the-loop gates, and the data model behind it all.	**You make it usable** Design chat flows people actually want to use, multilingual UX, technician journey on WhatsApp, change management, KPIs, and impact measurement.

What you'll take away: first-hand experience designing an agentic AI system for an industrial customer, a portfolio piece that spans AI, systems, and operations, and direct exposure to how world-class manufacturers think about technology.

Core features to design

These are the five pieces of the system. You don't need to build every one to completion, but your design must cover all of them.

1. Communication layer — WhatsApp-first

WhatsApp is the primary interface. Almost everyone in the factory already uses it. Your system should support:

- Request submission via text, voice note, or image.
- Real-time status updates back to the requestor.
- Direct technician communication inside the same thread.
- Multilingual conversation — English, French, and Hindi.

2. Intelligent request handling — the agent

When a request comes in, the AI agent should behave like the best maintenance dispatcher you've ever met:

- Ask clarifying questions until it understands the issue.
- Request additional inputs — a photo of the fault, a short video, a location.
- Identify the required skill — mechanic, electrician, welder, and so on.
- Assess urgency based on impact, using the priority tiers below.

Priority	Category	What it looks like
P0	**Production / safety critical**	A line is down, a safety hazard is present, a customer deadline is at risk.
P1	**Employee wellbeing**	Heat, lighting, ergonomics — issues that affect the people doing the work.
P2	**Non-urgent**	Cosmetic fixes, nice-to-haves, scheduled improvements.

3. Smart planning

- Propose schedules automatically — who goes where and when.
- Optimize technician allocation across open requests.
- Keep a human in the loop: the coordinator validates or adjusts.
- Adapt continuously as new requests arrive and priorities shift.

4. Technician workflow

- Receive the task on WhatsApp with the context the agent has already gathered.
- Ask the AI assistant questions — about the machine, history, parts, procedures.
- Start and stop tasks with timestamp and geolocation captured automatically.

- Upload completion proof — photos of the repair.

5. Feedback loop

- The requestor rates the completed work.
- That feedback feeds three things: technician performance, issue recurrence tracking, and quality-of-repair signals.
- Over time, the system gets smarter about which fixes stick and which don't.

End-to-end process flow

Six steps, one coherent journey:

1. **Request** — submitted via WhatsApp (text, voice, or image).
2. **Triage** — the AI agent clarifies, categorizes, and prioritizes.
3. **Schedule** — AI proposes an assignment; the coordinator validates.
4. **Dispatch** — the technician receives the task via WhatsApp.
5. **Execute** — work is tracked via geolocation, timestamps, and photos.
6. **Feedback** — the requestor evaluates; data feeds the learning loop.

Technical requirements

Your solution should address three layers:

AI capabilities

- A multimodal LLM that handles text, image, and voice.
- Conversational reasoning — follow-up, clarification, grounding.
- Task planning and orchestration across multiple tools and users.

System architecture

- A central relational database as the source of truth.
- Data persistence, backups, and auditability.
- API integrations — WhatsApp, location services, potentially IoT.

Data utilization

The system should enable the maintenance team to answer questions it can't easily answer today:

- Which failures are recurring, on which machines?
- Where is there a preventive maintenance opportunity hiding in the data?
- Which risks should we be watching?
- How is each technician performing against their peers?

Who uses the system

~20	~25	1	1
Requestors	**Technicians**	**Coordinator**	**Head of Maint.**
Report issues	Across mechanical, electrical, welding and more	Human-in-the-loop for planning	Oversight & strategy

Expected deliverables

Your submission should cover the five items below. A prototype is optional, but a strong one is a major advantage.

#	Deliverable	What to include
1	**Solution design**	System architecture, agent behavior, and the full set of user interaction flows.
2	**Prototype** *(optional)*	UI mockups or a working demo; sample chat interactions that show the agent in action.
3	**Decision logic**	How the agent prioritizes requests, how the planner optimizes technician assignments, and where humans step in.
4	**Data model**	What data is captured at every step, how it's structured, and how it gets reused for insight.
5	**Impact analysis**	Expected improvements vs. the current system — response time, machine uptime, coordination effort.

Evaluation criteria

The judges will weigh five things. Think of each as a question your submission has to answer convincingly:

Criterion	The question judges will ask
Practicality	Could this actually run on our factory floor next quarter?
Simplicity	Will a technician with no training adopt it on day one?
Innovation	Does it use agentic AI in a meaningful, non-gimmicky way?
Scalability	Does it still work at 10x the request volume or across more sites?
Impact	Can the improvements be measured — in minutes saved, uptime gained, or coordination removed?

What makes a winning solution

The strongest proposals tend to share four traits:

- **They remove human coordination, not just automate it.** The coordinator's job should shrink, not shift.
- **They show their reasoning.** Decision logic matters more than a polished UI.

- **They respect reality.** Patchy connectivity, informal language, users who don't read manuals — design for the real factory, not a demo.
- **They improve themselves.** Feedback loops should make the system noticeably better over six months.

Bonus extensions

If you have time and ambition, any of these will strengthen your proposal:

- Predictive maintenance using historical data.
- Integration with IoT / machine sensors.
- A voice-first interface for technicians on the floor.
- Offline-first capabilities for low-connectivity areas.
- Gamification of technician performance.

One last thing

This is not just a software challenge.

It's a systems thinking problem at the intersection of operations, human behavior, and AI decision-making.

Your goal is to design something people will actually use on a factory floor — not something that wins a slide, but something that wins a shift.